

PANCHAYAT SEVA AGENT

Complete System Blueprint — Version 3.0
Incorporating all AI Kyro Architecture Feedback

LLM Gemini 1.5 Pro	Voice Saaras + Bulbul	ReAct LangChain	Tools 10 Agent Tools	Notify WhatsApp API	Deploy AWS Mumbai
--------------------------	-----------------------------	--------------------	----------------------------	---------------------------	----------------------

AI Kyro Capstone • Thrissur, Kerala • 2025

1. What Changed in Version 3.0

This blueprint incorporates the full architecture review received from AI Kyro. Every change is listed below, with the reason why it was made. Nothing from v2 was removed — this is additive and refined.

Area	v2 (Old)	v3 (Updated — AI Kyro Feedback)
ReAct Orchestration	Custom Python regex parser	LangChain ReAct Agent — production-grade, no fragile regex
Reasoning LLM	Gemini 2.0 Flash	Gemini 1.5 Pro — better multi-step government service reasoning
Eligibility Logic	LLM-based (hallucination risk)	Rule-based Python (deterministic, accurate, no LLM needed)
Tools	9 tools	10 tools — Form Drafter Tool added
Speech Input	Basic Sarvam STT	Sarvam Saaras — dedicated Malayalam STT model
Speech Output	None	Sarvam Bulbul — Malayalam TTS for voice responses
Database	SQLite	PostgreSQL — multi-user, production-grade persistent storage
Knowledge Base	JSON file	ChromaDB vector store — semantic search over government docs
Web Scraping	DuckDuckGo search only	Playwright — scrapes Kerala gov portals directly
Notifications	Streamlit in-app banner	WhatsApp Business API — reaches users on their phones
Notification LLM	Gemini 1.5 Pro	Gemini Flash — faster and cheaper for alert text generation
Backend	Streamlit only	FastAPI backend + Streamlit / React admin dashboard
Deployment	Hugging Face Spaces	AWS Mumbai (ap-south-1) — data residency in India
User Onboarding	Form-style questions	Natural conversational onboarding — feels human, not bureaucratic
Evaluation	No formal dataset	150 Malayalam test queries + 30 user profiles + 5 metrics

2. Full System Overview

Panchayat Seva Agent is a two-part Agentic AI platform. The Reactive Agent answers any government service question in real time. The Proactive Scheduler runs every 24 hours and alerts users about new schemes before they even ask. Both subsystems share a persistent PostgreSQL user profile store.

✗ What This Is NOT (Chatbot)	☑ What This IS (True Agent)
Hardcoded if/else answers Only knows pre-programmed services User must always ask first Forgets everything between sessions English only — useless in rural Kerala Fails on any unexpected question	LLM reasons dynamically on every turn Handles ANY Kerala government service via web search Proactively alerts users about new schemes via WhatsApp Persistent user profiles in PostgreSQL Full Malayalam via Sarvam Saaras + Bulbul + Mayura LangChain ReAct — same architecture used in production AI

2.1 Two Subsystems

REACTIVE AGENT	Answers any government service question in Malayalam. Uses LangChain ReAct + Gemini 1.5 Pro + 10 tools. Works on demand, 24/7.
PROACTIVE SCHEDULER	Runs every 24h. Scrapes Kerala government portals using Playwright. Matches new schemes to user profiles using rule-based eligibility. Sends personalised Malayalam WhatsApp alerts.

3. Complete Technology Stack

Every technology choice is justified. All free tiers are available for development and demo day.

Layer	Technology	Free Tier	Why This Choice
AI Reasoning	Gemini 1.5 Pro	Yes — aistudio.google.com	Best multi-step reasoning for complex government queries
Alert Generation	Gemini Flash	Yes — same account	Faster and cheaper for short notification text
Malayalam STT	Sarvam Saaras	Yes — sarvam.ai	Dedicated Malayalam speech-to-text, highest accuracy

Malayalam TTS	Sarvam Bulbul	Yes — sarvam.ai	Natural Malayalam voice output for elderly users
Translation	Sarvam Mayura	Yes — sarvam.ai	EN→ML translation, used in final answer generation
ReAct Framework	LangChain	Yes — open source	Production-grade agent loop, no fragile regex parsing
Knowledge Base	ChromaDB	Yes — open source	Vector search over government PDFs and circulars
Web Scraping	Playwright	Yes — open source	Scrapes Kerala gov portals — more reliable than search
Database	PostgreSQL	Yes — Supabase free tier	Persistent user profiles and notifications
Scheduler	APScheduler	Yes — open source	Triggers proactive watcher every 24 hours
Notifications	WhatsApp Business API	Yes — Twilio sandbox	Reaches users on their phones, not just the app
Backend	FastAPI	Yes — open source	REST API layer between frontend and agent core
UI	Streamlit	Yes — open source	Chat interface — simple to build for demo
Deployment	AWS Mumbai	Free tier (12 months)	Data residency in India — Sovereign AI requirement

For demo day, Supabase free tier replaces a full AWS PostgreSQL setup. It gives you a real cloud PostgreSQL database with a dashboard, completely free. Sign up at supabase.com.

4. System Architecture — All Components

Component	Technology	Responsibility
Voice Input	Sarvam Saaras	Converts Malayalam speech to text on user's device
Streamlit UI	Streamlit	Chat interface — displays responses, notification banners
FastAPI Backend	FastAPI + Uvicorn	REST endpoints — /chat, /profile, /notifications
LangChain ReAct Agent	LangChain + Gemini 1.5 Pro	Thought→Tool→Observation loop, tool execution, final answer
Tool Suite	10 Python Functions	Web search, docs, office locator, eligibility, form drafter, memory, translation, voice, scraper, notifications
ChromaDB	ChromaDB	Vector store for semantic search over Kerala gov documents
Rule-Based Eligibility	Python	Deterministic income/age/category rules — no LLM guessing
PostgreSQL	Supabase / AWS RDS	Users, sessions, notifications, scheme_log tables
Scheme Watcher	Playwright + APScheduler	Scrapes Kerala gov portals every 24h for new schemes
Notifier	Python + Gemini Flash	Matches schemes to profiles, generates Malayalam alerts
WhatsApp Dispatcher	Twilio WhatsApp API	Sends personalized Malayalam messages to user phones
Sarvam Bulbul TTS	Sarvam API	Converts agent's text response to Malayalam audio
Voice Output	Browser Audio	Plays the Bulbul audio response in the Streamlit UI

5. All 10 Agent Tools

The LangChain ReAct agent autonomously decides which tools to call. Tools are Python functions wrapped as LangChain Tool objects. The agent never needs to be told which tool to use — it reasons about this itself.

5.1 Reactive Tools (used during user conversations)



Tool 1 — web_search(query)

Searches the internet for real-time information about any Kerala government service. Makes the agent capable of handling ANY query — not just pre-programmed ones. The most important tool.

INPUT: query: str — search terms in English

ACTION: Calls DuckDuckGo API (no key) or Serper API (better, free 2500/month). Returns top 5 results with title + snippet.



Tool 2 — chromadb_search(query)

Semantic search over locally ingested Kerala government documents, circulars, and PDFs. Faster and more authoritative than web search for well-known services.

INPUT: query: str — natural language query

ACTION: Embeds query with Gemini text-embedding-004. Searches ChromaDB collection. Returns top 3 matching document chunks.



Tool 3 — get_document_checklist(service, subtype)

Returns the structured document checklist for one of the 6 core services stored in the local knowledge base. Most reliable source for common services.

INPUT: service: str, subtype: str (e.g. 'newborn', 'bpl', 'general')

ACTION: Looks up knowledge.json. Falls back to web_search if service unknown. Returns documents, office, fee, timeline, portal link.



Tool 4 — find_office(service, district)

Finds the correct government office for a service — name, address, counter, phone, hours. Prevents wasted trips to the wrong office.

INPUT: service: str, district: str

ACTION: Queries local office database + Playwright scrape of lsgkerala.gov.in for up-to-date office listings.



Tool 5 — check_eligibility(scheme_name, user_profile) ★ NEW

Rule-based eligibility check — deterministic Python logic, no LLM involved. Prevents hallucinated eligibility decisions for critical welfare schemes.

INPUT: scheme_name: str, user_profile: dict

ACTION: Loads eligibility rules from eligibility_rules.json. Applies income/age/category/family_size filters. Returns eligible: true/false + reason.



Tool 6 — draft_form(service, user_profile) ★ NEW ★ NEW

Drafts a pre-filled application form content in Malayalam based on the user's profile and the selected service. The user reviews and takes it to the office.

INPUT: service: str, user_profile: dict

ACTION: Calls Gemini 1.5 Pro with a structured form-drafting prompt. Returns a formatted Malayalam draft with all fields filled from the user profile.



Tool 7 — translate_to_malayalam(text)

Translates any English text to fluent, natural Malayalam using Sarvam Mayura. Called as the final step before every response to a Malayalam-speaking user.

INPUT: text: str — English content to translate

ACTION: Calls Sarvam AI /translate endpoint with model=mayura:v1. Falls back to returning English if API times out.



Tool 8 — save_to_memory(key, value)

Saves a fact about the user to both the session memory dict and the PostgreSQL user profile. The agent never asks for information it already has.

INPUT: key: str, value: str

ACTION: Writes to in-memory dict + calls PostgreSQL update via psycopg2. Returns confirmation string.



Tool 9 — get_from_memory(key)

Retrieves a saved fact about the user. Called at the start of every turn before asking any clarifying question.

INPUT: key: str — the fact to retrieve

ACTION: Checks session memory dict first (fast), then PostgreSQL fallback. Returns value or 'not_found'.

5.2 Proactive Tools (used by Scheme Watcher only)



Tool 10 — scrape_kerala_gov()

Scrapes Kerala government portals directly using Playwright. More reliable than web search for official scheme announcements. Targets: kerala.gov.in, lsgkerala.gov.in, social justice dept pages.

INPUT: None — runs on scheduler

ACTION: Playwright headless browser. Extracts news/announcement sections. Returns raw HTML parsed to text. De-duplicates against scheme_log table.

6. The LangChain ReAct Loop

In v3, the custom regex-based agent.py is replaced with LangChain's built-in ReAct agent. LangChain handles all the Thought→Action→Observation parsing, tool routing, and loop management. Your job is just to define the tools and the system prompt.

T
THOUGHT

User in Thrissur wants to open a food stall. I have district in memory already. This requires: (1) FSSAI food license — will check eligibility first since it's income-dependent, (2) Thrissur Corporation trade license. Let me research both in sequence.

A
ACTION

```
check_eligibility({"scheme_name": "FSSAI Basic Registration", "user_profile": {"income": 80000, "business_type": "food_stall"}}) find_office({"service": "trade_license", "district": "Thrissur"})
```

O
OBSERVATION

FSSAI Basic: ELIGIBLE — turnover < Rs.12L/year qualifies for basic registration, fee Rs.100/year. | Trade License: Thrissur Corporation, Town Hall Road, Counter 3, 0487-2360028, Mon–Fri 10am–4pm. Documents: premises proof, Aadhaar, application form.

A
ACTION

```
draft_form({"service": "fssai_basic", "user_profile": {"name": "Rajan", "district": "Thrissur", "business": "tea stall"}})
```

O
OBSERVATION

Form draft generated: Name: Rajan | Business Type: Food Stall | Address: [user to fill] | Nature of food: Tea, snacks | Annual turnover estimate: < Rs. 12 lakh | Signature: [user to sign]

T
THOUGHT

I have eligibility confirmation, office details, and a form draft for FSSAI. I have office details for trade license. I have enough to give a complete, helpful Malayalam response. Let me translate and send.

✓
FINAL
ANSWER

translate_to_malayalam(complete_step_by_step_plan) → Send formatted Malayalam response with: FSSAI eligibility confirmed, form draft attached, trade license office details, both fees, both timelines, pro tip to go to FSSAI first.

7. Complete Project File Structure

panchayat-seva-agent/ — every file and its purpose

```

app.py                ← Streamlit chat UI with voice + notification banners main.py                ←
FastAPI backend — /chat, /profile, /notify endpoints agent.py                ← LangChain ReAct
agent setup + tool registration tools.py                ← All 10 tool functions eligibility.py                ←
Rule-based eligibility checker (deterministic Python) form_drafter.py                ← Form drafting
logic + Gemini prompt memory.py                ← Session memory + PostgreSQL profile manager
chromadb_ingest.py    ← One-time script to ingest gov docs into ChromaDB
scheme_watcher.py     ← Playwright scraper + APScheduler 24h job notifier.py                ←
Eligibility matcher + WhatsApp dispatcher database.py                ← PostgreSQL table setup
(users, notifications, scheme_log) prompts.py                ← System prompt, onboarding script,
ReAct format rules knowledge.json                ← 6 core service document checklists
eligibility_rules.json ← All scheme eligibility rules (income, age, category) requirements.txt
← All Python dependencies .env                ← API keys — NEVER commit to GitHub
  
```

8. Implementation — Every File

File 1 — requirements.txt

```
streamlit==1.32.0
fastapi==0.110.0
uvicorn==0.27.0
langchain==0.1.12
langchain-google-genai==1.0.1
google-generativeai==0.5.0
chromadb==0.4.24
playwright==1.42.0
psycpg2-binary==2.9.9
apscheduler==3.10.4
twilio==8.5.0
requests==2.31.0
python-dotenv==1.0.0
duckduckgo-search==5.3.0
```

File 2 — .env (Never Commit This)

```
GEMINI_API_KEY=your_gemini_key
SARVAM_API_KEY=your_sarvam_key
TWILIO_ACCOUNT_SID=your_twilio_sid
TWILIO_AUTH_TOKEN=your_twilio_token
TWILIO_WHATSAPP_FROM=whatsapp:+14155238886
DATABASE_URL=postgresql://user:pass@db.supabase.co:5432/postgres
```

File 3 — eligibility_rules.json (Rule-Based — Critical)

This is the most important design decision from AI Kyro feedback: eligibility must NEVER be decided by an LLM. It must be deterministic rules. A wrong LLM answer could send someone to apply for a scheme they don't qualify for — wasting their time. Rules live here.

```
{
  // Each rule has: conditions (AND logic) + disqualifiers (any one fails =
  ineligible)
  "bpl_ration_card"
: {
  "conditions": {
    "annual_income_max": 100000,
    "must_have_aadhaar": true,
    "kerala_resident": true
  },
  "disqualifiers": ["has_apl_card", "income_above_100000"],
  "benefit": "Free ration - rice, wheat, sugar per month"
},
  "pm_awas_gramin"
: {
  "conditions": {
```

```

    "annual_income_max": 300000,
    "has_pucca_house": false,
    "kerala_resident": true
  },
  "disqualifiers": ["has_pucca_house", "income_above_300000"],
  "benefit": "Rs.1.2 lakh housing grant"
},
"karunya_health":
: {
  "conditions": {"annual_income_max": 300000, "is_bpl": true},
  "disqualifiers": ["has_private_insurance"],
  "benefit": "Up to Rs.2 lakh medical assistance"
},
"old_age_pension":
: {
  "conditions": {"age_min": 60, "annual_income_max": 100000,
"benefit": "Rs.1600/month pension"
}

```

File 4 — eligibility.py (Deterministic Rule Engine)

Pure Python logic. No LLM calls. Returns a clear eligible/not-eligible decision with a human-readable reason. This is what the `check_eligibility` tool calls.

```

# eligibility.py - rule-based, no LLM, no hallucinations
import json

with open('eligibility_rules.json') as f:
    RULES = json.load(f)

def check_eligibility(scheme_name: str, user_profile: dict) -> dict:
    """
    Returns: { eligible: bool, reason: str, benefit: str }
    Pure rule-based - deterministic, no LLM involved.
    """
    key = scheme_name.lower().replace(' ', '_')
    if key not in RULES:
        return {'eligible': None, 'reason': 'Scheme not in local rules - use
web_search for details.'}

    rule = RULES[key]
    conds = rule.get('conditions', {})
    disq = rule.get('disqualifiers', [])

    # Check disqualifiers first - any one fails = not eligible
    for d in disq:
        if user_profile.get(d, False):
            return {'eligible': False, 'reason': f'Not eligible:
{d.replace("_", " ")}'},
                    'benefit': rule.get('benefit', '')}

```

```

# Check all conditions
for cond, val in conds.items():
    if cond.endswith('_max'):
        field = cond.replace('_max', '')
        if user_profile.get(field, 0) > val:
            return {'eligible': False,
                    'reason': f'{field} Rs.{user_profile.get(field)}
exceeds limit Rs.{val}',
                    'benefit': rule.get('benefit', '')}
    elif cond.endswith('_min'):
        field = cond.replace('_min', '')
        if user_profile.get(field, 0) < val:
            return {'eligible': False,
                    'reason': f'{field} {user_profile.get(field)} is
below minimum {val}',
                    'benefit': rule.get('benefit', '')}
    elif isinstance(val, bool):
        if user_profile.get(cond, not val) != val:
            return {'eligible': False, 'reason': f'Condition not met:
{cond}',
                    'benefit': rule.get('benefit', '')}

    return {'eligible': True,
            'reason': 'All conditions met',
            'benefit': rule.get('benefit', 'Benefit details available at the
office')}

```

File 5 — form_drafter.py (New Form Drafter Tool)

Important disclaimer from AI Kyro: Always label drafted forms as templates for the user to review. Never present them as guaranteed valid legal documents. The form_drafter adds this disclaimer automatically.

```

# form_drafter.py - NEW in v3
import os
import google.generativeai as genai

genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
model = genai.GenerativeModel('gemini-1.5-pro')

FORM_TEMPLATES = {
    'birth_certificate': ['Child name', 'Date of birth', 'Place of
birth', 'Father name', 'Mother name', 'Hospital', 'Ward/Village'],
    'ration_card': ['Applicant name', 'Address', 'Family
members', 'Income', 'Aadhaar numbers', 'Existing card number'],
    'trade_license': ['Business name', 'Owner name', 'Business
address', 'Type of trade', 'Area sq.ft', 'Aadhaar'],
    'fssai_basic': ['Business name', 'Owner name', 'Address', 'Food
category', 'Annual turnover estimate', 'Aadhaar'],
}

DISCLAIMER = (
    '\n\n--- IMPORTANT ---\n'

```

```

    'This is a DRAFT TEMPLATE only. Review all fields carefully before
    submitting.\n'
    'Consult the Akshaya Centre or a government official to verify before
    filing.\n'
    'This draft does not guarantee acceptance by any government authority.'
)

def draft_form(service: str, user_profile: dict) -> str:
    fields = FORM_TEMPLATES.get(service,
    ['Name', 'Address', 'Aadhaar', 'District'])
    fields_str = ', '.join(fields)
    profile_str = str(user_profile)

    prompt = f'''
    Draft a pre-filled application form for: {service}
    Required fields: {fields_str}
    User profile data: {profile_str}
    Fill in all fields you can from the profile data.
    Leave [USER TO FILL] for any field not in the profile.
    Format as a clean, readable form in English.
    Do not invent data. Only use what is in the profile.
    '''

    response = model.generate_content(prompt)
    return response.text + DISCLAIMER

```

File 6 — agent.py (LangChain ReAct — Replaces Custom Parser)

LangChain wraps your tools, manages the ReAct loop, and handles all parsing. You only need to define tools as LangChain Tool objects and call `initialize_agent`.

```

# agent.py — LangChain ReAct (v3)
import os
from langchain.agents import initialize_agent, AgentType
from langchain.tools import Tool
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.memory import ConversationBufferMemory
from tools import (web_search, chromadb_search, get_document_checklist,
                  find_office, translate_to_malayalam)
from eligibility import check_eligibility
from form_drafter import draft_form
from memory import save_to_memory, get_from_memory
from prompts import SYSTEM_PROMPT

# Wrap each function as a LangChain Tool
tools = [
    Tool(name='web_search', func=web_search,
        description='Search internet for Kerala government service info.
        Input: search query string.'),
    Tool(name='chromadb_search', func=chromadb_search,
        description='Semantic search over Kerala government documents. Input:
        natural language query.'),

```

```

    Tool(name='get_document_checklist', func=lambda x:
get_document_checklist(*x.split('|'))),
        description='Get document list for a known service. Input:
service_name|subtype (e.g. birth_certificate|newborn).'),
    Tool(name='find_office', func=lambda x:
find_office(*x.split('|'))),
        description='Find government office address and hours. Input:
service_name|district.'),
    Tool(name='check_eligibility', func=lambda x:
str(check_eligibility(*eval(x))),
        description='Rule-based eligibility check. Input: ["scheme_name",
{user_profile_dict}].'),
    Tool(name='draft_form', func=lambda x: draft_form(*eval(x)),
        description='Draft a pre-filled Malayalam application form. Input:
["service", {user_profile}].'),
    Tool(name='translate_to_malayalam', func=translate_to_malayalam,
        description='Translate English text to Malayalam. Input: English text
string.'),
    Tool(name='save_to_memory', func=lambda x:
save_to_memory(*x.split('|'))),
        description='Save a user fact. Input: key|value (e.g.
district|Thrissur).'),
    Tool(name='get_from_memory', func=get_from_memory,
        description='Retrieve a saved user fact. Input: key string.'),
]

# LangChain LLM - Gemini 1.5 Pro
llm = ChatGoogleGenerativeAI(
    model='gemini-1.5-pro',
    google_api_key=os.getenv('GEMINI_API_KEY'),
    temperature=0.1
)

# Session memory (holds full conversation history)
memory = ConversationBufferMemory(memory_key='chat_history',
return_messages=True)

# Initialize LangChain ReAct agent
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.CHAT_CONVERSATIONAL_REACT_DESCRIPTION,
    memory=memory,
    verbose=True,
    max_iterations=10,
    agent_kwargs={'system_message': SYSTEM_PROMPT}
)

def run_agent(user_message: str) -> str:
    try:
        return agent.run(user_message)
    except Exception as e:
        return f'Error: {str(e)}. Please try again.'
```

File 7 — scheme_watcher.py (Playwright Scraper + Scheduler)

Playwright scrapes Kerala government portals directly — much more reliable than web search for official announcements. APScheduler triggers it every 24 hours.

```
# scheme_watcher.py — Playwright scraper
import os, json, hashlib
from datetime import datetime
from playwright.sync_api import sync_playwright
from apscheduler.schedulers.background import BackgroundScheduler
import google.generativeai as genai
from notifier import notify_eligible_users
from database import get_db_connection

SCRAPE_TARGETS = [
    'https://kerala.gov.in/en/web/guest/news',
    'https://lsgkerala.gov.in/en/news',
    'https://socialsecuritymission.gov.in/scheme',
]

def scrape_portal(url: str) -> str:
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()
        page.goto(url, timeout=30000)
        text = page.inner_text('body')
        browser.close()
        return text[:5000] # limit to first 5000 chars

def parse_schemes(raw_text: str) -> list:
    '''Use Gemini Flash — fast and cheap for parsing'''
    genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
    model = genai.GenerativeModel('gemini-1.5-flash')
    prompt = f'''Extract government scheme announcements from this text.
Return JSON array. Each item: name, summary, eligibility, deadline, link.
Return only valid JSON array, nothing else.\n\nText:\n{raw_text}'''
    try:
        resp = model.generate_content(prompt)
        return json.loads(resp.text.strip().strip('```json').strip('```'))
    except:
        return []

def is_new_scheme(name: str) -> bool:
    sid = hashlib.md5(name.encode()).hexdigest()
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('SELECT 1 FROM scheme_log WHERE scheme_id=%s', (sid,))
    exists = cur.fetchone()
    if not exists:
        cur.execute('INSERT INTO scheme_log VALUES (%s,%s,%s)',
            (sid, name, datetime.now().isoformat()))
        conn.commit()
    cur.close(); conn.close()
    return not exists
```

```
def watch_for_new_schemes():
    print(f'[{datetime.now()}] Scheme Watcher running...')
    for url in SCRAPE_TARGETS:
        try:
            raw = scrape_portal(url)
            schemes = parse_schemes(raw)
            for s in schemes:
                if s.get('name') and is_new_scheme(s['name']):
                    print(f' New scheme: {s["name"]}')
                    notify_eligible_users(s)
        except Exception as e:
            print(f' Error scraping {url}: {e}')

def start_scheduler():
    scheduler = BackgroundScheduler()
    # Run every day at 8am
    scheduler.add_job(watch_for_new_schemes, 'cron', hour=8, minute=0)
    scheduler.start()
    print('Scheme Watcher scheduler started.')
    return scheduler
```

File 8 — notifier.py (WhatsApp Dispatcher)

Matches new schemes to eligible users using the rule-based eligibility engine. Generates personalised Malayalam messages using Gemini Flash. Sends via Twilio WhatsApp API.

```
# notifier.py - WhatsApp notification dispatcher
import os
from twilio.rest import Client
import google.generativeai as genai
from eligibility import check_eligibility
from tools import translate_to_malayalam
from database import get_db_connection

def get_all_users() -> list:
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('SELECT
user_id,name,phone,district,income,family_size,category,age,notify FROM
users WHERE notify=true')
    cols =
['user_id','name','phone','district','income','family_size','category','age',
,'notify']
    rows = [dict(zip(cols,r)) for r in cur.fetchall()]
    cur.close(); conn.close()
    return rows

def generate_alert_text(user: dict, scheme: dict) -> str:
    '''Gemini Flash - fast + cheap for short text generation'''
    genai.configure(api_key=os.getenv('GEMINI_API_KEY'))
    model = genai.GenerativeModel('gemini-1.5-flash')
    prompt = f'''
Write a short, warm WhatsApp notification in English for:
```



```

    User: {user['name']} from {user['district']}
    Scheme: {scheme['name']}
    Benefit: {scheme.get('summary','')}
    Deadline: {scheme.get('deadline','Check portal')}
    Link: {scheme.get('link','kerala.gov.in')}
    Keep it under 100 words. Warm and encouraging tone.
    Tell them to visit their nearest Akshaya Centre for help applying.
    '''

    resp = model.generate_content(prompt)
    en_msg = resp.text
    return translate_to_malayalam(en_msg)

def send_whatsapp(phone: str, message: str):
    client = Client(os.getenv('TWILIO_ACCOUNT_SID'),
                    os.getenv('TWILIO_AUTH_TOKEN'))
    client.messages.create(
        body=message,
        from_=os.getenv('TWILIO_WHATSAPP_FROM'),
        to=f'whatsapp:{phone}'
    )

def notify_eligible_users(scheme: dict):
    for user in get_all_users():
        profile = {k: user.get(k) for k in
['income', 'family_size', 'category', 'age']}
        result = check_eligibility(scheme['name'], profile)
        if result.get('eligible'):
            msg = generate_alert_text(user, scheme)
            if user.get('phone'):
                send_whatsapp(user['phone'], msg)
                print(f'  WhatsApp sent to {user["name"]} about
{scheme["name"]}')

```

File 9 — prompts.py (Natural Onboarding Conversation)

AI Kyro specifically flagged: profile creation must feel like a conversation, not a form. The agent learns district, income, family size naturally through conversation — never by showing a form.

```

# The onboarding sequence — agent asks these one by one, naturally
# DO NOT ask all at once. One question per turn.

ONBOARDING_SCRIPT
= [
    "Namaskaram! Ente peru Panchayat Seva Agent. Ningalku enthu sahaayam vendum?",
    "Njan ninnakayi sheri aaya office kaaNicchu tharaan, ningal ethu jillayilaanu?",
    "Ningalude pera enthu? Ellaa communications-um personalized aakkaan.",
    "Sarkaar schemes eligibility check cheyyaan — kudum-batthile aaLukaL ethra?",
    "Varsham varumanam approximately ethra? (Approximate mathi — exact venda)",
]

```

```
# Main system prompt – see previous blueprint for full version
# Key addition in v3: LangChain-compatible format
SYSTEM_PROMPT = '''
You are Panchayat Seva Agent – expert AI for Kerala government services in Malayalam.
You use tools to answer ANY question. You never make up information.
Always call translate_to_malayalam as your final step for Malayalam users.
Always call get_from_memory at the start of each turn before asking any question.
For scheme eligibility: ALWAYS use check_eligibility tool – never guess.
For form drafting: ALWAYS add the disclaimer that the output is a template only.
When district is unknown: ask conversationally, then save_to_memory.
'''
```

File 10 — app.py (Full Streamlit UI with Voice + Notifications)

```
# app.py – Full UI with voice input, TTS output, notification banners
import streamlit as st
import requests as req
import uuid, os
from database import get_unread_notifications, mark_read
from scheme_watcher import start_scheduler
from database import init_db

init_db()
st.set_page_config(page_title='Panchayat Seva Agent', page_icon='🏠',
layout='wide')

# Start background scheduler once per session
if 'scheduler' not in st.session_state:
    st.session_state.scheduler = start_scheduler()

# Assign session user ID
if 'user_id' not in st.session_state:
    st.session_state.user_id = str(uuid.uuid4())

st.title('🏠 Panchayat Seva Agent')
st.caption('Kerala Government Services – Malayalam Voice & Text')

# Show unread WhatsApp-style notification banners
for n in get_unread_notifications(st.session_state.user_id):
    st.info(f'🔔 {n["scheme_name"]}\n\n{n["message_ml"]}')
    mark_read(n['id'])

# Voice input toggle
use_voice = st.sidebar.toggle('🔊 Voice Input (Malayalam)', value=False)

if 'messages' not in st.session_state: st.session_state.messages = []
```

```
for msg in st.session_state.messages:
    with st.chat_message(msg['role']): st.write(msg['content'])

if prompt := st.chat_input('Type in Malayalam or English...'):
    st.session_state.messages.append({'role': 'user', 'content': prompt})
    with st.chat_message('user'): st.write(prompt)
    with st.chat_message('assistant'):
        with st.spinner('ചിന്തിക്കുന്നു... / Thinking...'):
            resp = req.post('http://localhost:8000/chat',
                            json={'message': prompt,
                                    'user_id': st.session_state.user_id})
            reply = resp.json().get('reply', 'Error. Please retry.')
            st.write(reply)
        st.session_state.messages.append({'role': 'assistant', 'content': reply})
```

9. Evaluation Dataset (AI Kyro Requirement)

AI Kyro specifically asked for 150 test queries and 5 evaluation metrics. This is what separates a hobby project from a professional one. Build this dataset in Week 3.

9.1 150 Malayalam Test Queries — Category Breakdown

Category	Query Count	Example Queries
Document Services (birth, death, ration, land, caste, building)	40	Ente makalkku janana certificate vendum - Ration card renew cheyyaan enthu vendum
Scheme Eligibility (housing, pension, health, scholarship)	35	'PM Awas scheme-il eligibility undennikku?', 'Karunya scheme enthokke tharum?'
Business Permits (FSSAI, trade, GST, shop license)	30	'Chaya kada thuranganam. Enthu license vendum?', 'FSSAI registration ethaanu?'
Multi-service Workflows (goals needing 3+ services)	25	'Oru medical shop thudangaanm, enthu cheyyaanam?'
RTI Procedures	10	'Oru RTI application ezhuthaanm, ethaanu format?'
Edge Cases & Fallbacks	10	Gibberish, out-of-scope queries, missing information scenarios
TOTAL	150	

Validate correct answers with 2 panchayat officers or Akshaya centre staff. Their sign-off makes your evaluation dataset credible to judges and real-world stakeholders.

9.2 5 Evaluation Metrics

Metric	Target	How to Measure
ReAct Answer Accuracy	≥ 85% correct	Compare agent answer to ground truth validated by experts
Tool Call Appropriateness	≥ 80% correct tools	Did the agent call web_search when it should? draft_form when asked?
Proactive Match Precision	≥ 90% correct matches	Of notifications sent, how many were to actually eligible users?
Malayalam Language Quality	≥ 4.0 / 5.0	Native Malayalam speaker rates fluency + naturalness of responses
End-to-End Response Time	≤ 8 seconds	Time from user message to final Malayalam response appearing

10. Akshaya Centre Integration Strategy

AI Kyro specifically suggested Akshaya Centre integration as a real-world validation. Kerala has 3,000+ Akshaya Centres across all districts — they serve exactly your target population and already have digital infrastructure.

Why This Matters for Demo Day

Telling a judge 'I piloted this at an Akshaya Centre in Thrissur' is worth more than any slide. It means real users, real feedback, and real validation. Even one centre is enough. Do this before April 5th.

10.1 How to Approach an Akshaya Centre

1. **Find your nearest Akshaya Centre** — akshaya.kerala.gov.in has the directory.
2. **Visit or WhatsApp the centre manager. Say: 'I am a student building a Malayalam AI assistant for government services. Can I test it with a few of your customers? It takes 10 minutes.'**
3. **Run 3–5 live test sessions. Record (with permission) the user interacting with the bot.**
4. **Collect feedback: Was it helpful? Was the Malayalam natural? Was anything missing?**
5. **Use the video and feedback in your Demo Day pitch. This is your strongest possible evidence.**

10.2 Integration in the Agent

Add Akshaya Centre locations to the `find_office` tool output. When a user needs help applying, the agent should always say: 'Your nearest Akshaya Centre is at [address] — they can help you fill this form for free.' This makes the agent immediately more practical.

11. Four-Week Build Roadmap

Week	Theme	Daily Tasks
Week 1 Mar 8–14	Foundation	Day 1–2: Get all API keys (Gemini, Sarvam, Twilio, Supabase). Set up PostgreSQL via Supabase. Build <code>database.py</code> . Day 3–4: Write <code>eligibility_rules.json</code> for 8 schemes. Build <code>eligibility.py</code> . Write unit tests for all rules. Day 5–7: Build <code>tools.py</code> (<code>web_search</code> ,

		translate_to_malayalam). Write knowledge.json for 6 services. Test both APIs manually.
Week 2 Mar 15–22	Reactive Agent	Day 1–2: Write prompts.py with full system prompt. Install LangChain. Build agent.py with all 9 reactive tools registered. Day 3–4: Build FastAPI backend (main.py). Connect Streamlit to FastAPI. Test 20 queries end-to-end. Day 5–7: Add form_drafter.py. Add chromadb_ingest.py (ingest 5 Kerala gov PDFs). Test form drafting flow.
Week 3 Mar 23–31	Proactive System	Day 1–3: Build scheme_watcher.py with Playwright. Test scraping on all 3 portal URLs. Build notifier.py. Day 4–5: Integrate Twilio WhatsApp sandbox. Test full flow: new scheme → eligibility match → WhatsApp message. Day 6–7: Add Sarvam Saaras STT and Bulbul TTS to app.py. Visit Akshaya Centre for first user test.
Week 4 Apr 1–5	Polish + Demo	Day 1: Run all 150 test queries. Log accuracy, fix bugs. Day 2: Deploy to AWS Mumbai (or Supabase + Streamlit Cloud as backup). Day 3: Record 3-minute demo video showing both subsystems. Day 4: Build 5-slide pitch deck. Include Akshaya Centre pilot feedback. Day 5: Final rehearsal. Test on mobile. Confirm live URL works.

12. Demo Day — The Winning Strategy

12.1 3-Minute Demo Script

6. **0:00–0:20 The Hook** — One real story. 'A family in Mukundapuram qualified for PM Awas Yojana housing grant. They found out 4 months after the deadline. This system would have WhatsApp'd them in Malayalam the day it was announced.' Open the live app.
7. **0:20–1:00 Proactive Demo** — Show a notification banner. Say: 'At 8am today, our Scheme Watcher scraped Kerala government portals, found this new scheme, matched it to our user profile using rule-based eligibility — zero LLM guessing — and sent this WhatsApp in Malayalam. Automatically. Nobody asked.'
8. **1:00–2:00 Reactive Demo** — Have a real person (ideally from the Akshaya Centre) type a question in Malayalam. Show the LangChain ReAct loop running — each tool call visible. Show the agent handling a service it was never explicitly programmed for.
9. **2:00–2:30 Technical Depth** — One slide: LangChain ReAct + Gemini 1.5 Pro + Sarvam Saaras/Bulbul + rule-based eligibility + Playwright + WhatsApp. Then say: 'Eligibility checks are deterministic rule-based Python — zero chance of the AI sending an alert to someone who does not qualify.'
10. **2:30–3:00 Scale** — 'Kerala has 941 Gram Panchayats. 3.5 crore citizens. 3,000 Akshaya Centres. This system is ready to deploy to all of them tomorrow, for free, in Malayalam — and in 10 other Indian languages through Sarvam AI's multilingual stack.'

12.2 Five Slides

- Slide 1 — The Problem: One image. One story. Three lines. No jargon.
- Slide 2 — The Solution: Screenshot of Malayalam conversation. Screenshot of WhatsApp notification.
- Slide 3 — Architecture: Two-part system diagram. ReAct loop + Proactive scheduler.
- Slide 4 — Technical Choices: Why LangChain, why Sarvam, why rule-based eligibility, why Playwright.
- Slide 5 — Validation + Scale: Akshaya Centre pilot results. 941 Panchayats. Sovereign AI.

The One Thing That Wins

Bring a real person from the Akshaya Centre pilot. Let them speak to the agent in Malayalam, live, in front of the judges. Then show a WhatsApp notification arriving on your phone from the proactive system. When a judge sees real human impact — an elderly Malayalam speaker genuinely getting help — and a WhatsApp alert arriving automatically — no slide deck, no competitor, no amount of technical sophistication can beat that moment. That is what Sovereign AI means.

Panchayat Seva Agent — Blueprint v3.0

LangChain • Gemini 1.5 Pro • Sarvam AI • WhatsApp • AWS Mumbai

AI Kyro Capstone 2025 • Thrissur, Kerala • Sovereign AI